

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

15

Query Planning & Optimization – Part 2



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Project #3 will be released this week.
It is due Sun Nov 17th @ 11:59pm.

Homework #4 will be released next week.
It is due Wed Nov 13th @ 11:59pm.



QUERY OPTIMIZATION

Heuristics / Rules

- Rewrite the query to remove stupid / inefficient things.
- These techniques may need to examine catalog, but they do not need to examine data.

Cost-based Search

- Use a model to estimate the cost of executing a plan.
- Evaluate multiple equivalent plans for a query and pick the one with the lowest cost.

TODAY'S AGENDA

Plan Cost Estimation

Plan Enumeration

Nested Sub-queries



COST ESTIMATION

How long will a query take?

- CPU: Small cost; tough to estimate
- Disk: # of block transfers
- Memory: Amount of DRAM used
- Network: # of messages

How many tuples will be read/written?

It is too expensive to run every possible plan to determine this information, so the DBMS need a way to derive this information...

STATISTICS

The DBMS stores internal statistics about tables, attributes, and indexes in its internal catalog.

Different systems update them at different times.

Manual invocations:

- Postgres/SQLite: **ANALYZE**
- Oracle/MySQL: **ANALYZE TABLE**
- SQL Server: **UPDATE STATISTICS**
- DB2: **RUNSTATS**



STATISTICS

For each relation **R**, the DBMS maintains the following information:

- N_R : Number of tuples in **R**.
- $V(A, R)$: Number of distinct values for attribute **A**.



DERIVABLE STATISTICS

The selection cardinality $SC(A, R)$ is the average number of records with a value for an attribute A given $N_R / V(A, R)$

Note that this assumes *data uniformity*.

→ 10,000 students, 10 colleges – how many students in SCS?

SELECTION STATISTICS

Equality predicates on unique keys are easy to estimate.

```
SELECT * FROM people  
WHERE id = 123
```

What about more complex predicates? What is their selectivity?

```
SELECT * FROM people  
WHERE val > 1000
```

```
SELECT * FROM people  
WHERE age = 30  
AND status = 'Lit'
```

```
CREATE TABLE people (  
  id INT PRIMARY KEY,  
  val INT NOT NULL,  
  age INT NOT NULL,  
  status VARCHAR(16)  
);
```

COMPLEX PREDICATES

The selectivity (**sel**) of a predicate **P** is the fraction of tuples that qualify.

Formula depends on type of predicate:

- Equality
- Range
- Negation
- Conjunction
- Disjunction



COMPLEX PREDICATES

The selectivity (**sel**) of a predicate **P** is the fraction of tuples that qualify.

Formula depends on type of predicate:

- Equality
- Range
- Negation
- Conjunction
- Disjunction

SELECTIONS – COMPLEX PREDICATES

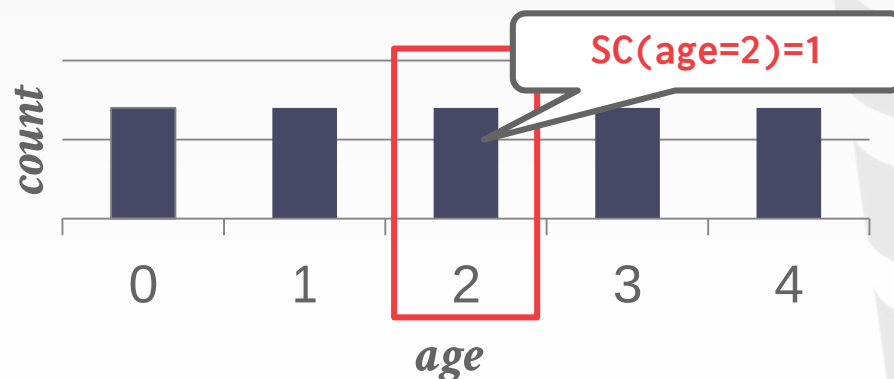
Assume that $V(\text{age}, \text{people})$ has five distinct values (0–4) and $N_R = 5$

Equality Predicate: $A = \text{constant}$

→ $\text{sel}(A = \text{constant}) = SC(P) / N_R$

→ Example: $\text{sel}(\text{age} = 2) = 1/5$

```
SELECT * FROM people
WHERE age = 2
```



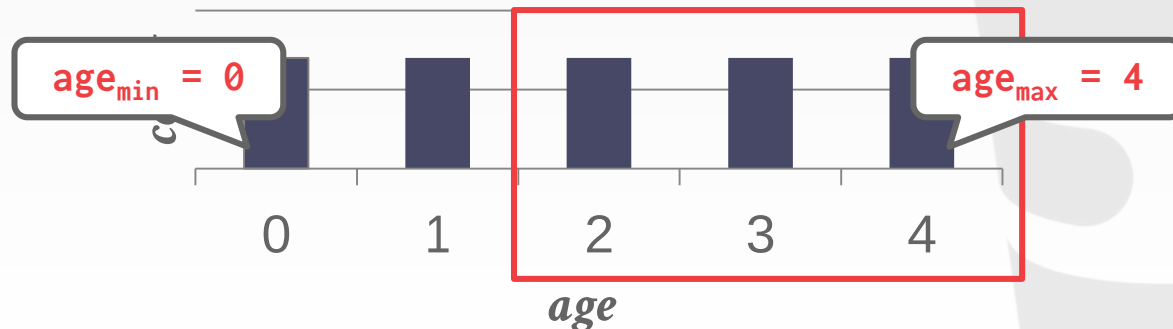
SELECTIONS – COMPLEX PREDICATES

Range Predicate:

→ $\text{sel}(A \geq a) = (A_{\max} - a) / (A_{\max} - A_{\min})$

→ Example: $\text{sel}(\text{age} \geq 2) \approx (4 - 2) / (4 - 0)$
 $\approx 1/2$

```
SELECT * FROM people  
WHERE age >= 2
```



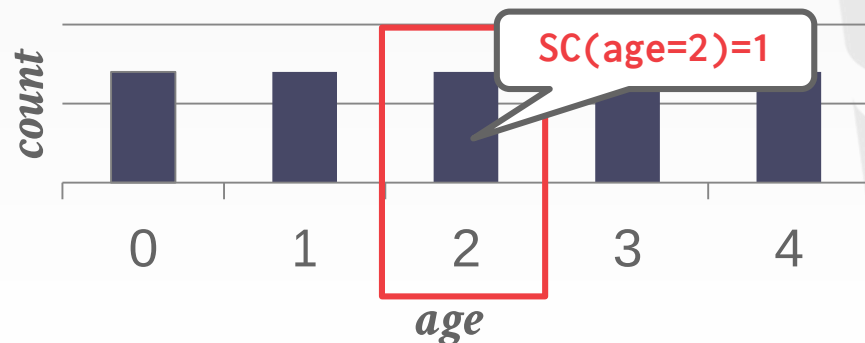
SELECTIONS – COMPLEX PREDICATES

Negation Query:

→ $\text{sel}(\text{not } P) = 1 - \text{sel}(P)$

→ Example: $\text{sel}(\text{age} \neq 2)$

```
SELECT * FROM people  
WHERE age != 2
```



SELECTIONS – COMPLEX PREDICATES

Negation Query:

→ $\text{sel}(\text{not } P) = 1 - \text{sel}(P)$

→ Example: $\text{sel}(\text{age} \neq 2) = 1 - (1/5) = 4/5$

```
SELECT * FROM people  
WHERE age != 2
```

Observation: Selectivity $\approx$ Probability



SELECTIONS – COMPLEX PREDICATES

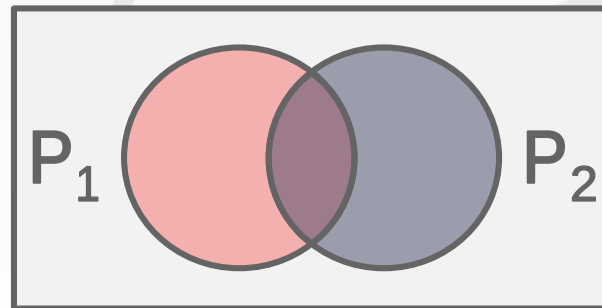
Conjunction:

→ $\text{sel}(P_1 \wedge P_2) = \text{sel}(P_1) \cdot \text{sel}(P_2)$

→ $\text{sel}(\text{age}=2 \wedge \text{name LIKE 'A\%'})$

This assumes that the predicates are independent.

```
SELECT * FROM people
WHERE age = 2
      AND name LIKE 'A%'
```



SELECTIONS – COMPLEX PREDICATES

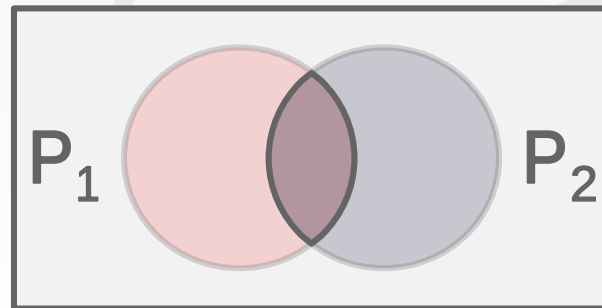
Conjunction:

→ $\text{sel}(P1 \wedge P2) = \text{sel}(P1) \cdot \text{sel}(P2)$

→ $\text{sel}(\text{age}=2 \wedge \text{name LIKE 'A\%'})$

This assumes that the predicates are independent.

```
SELECT * FROM people  
WHERE age = 2  
      AND name LIKE 'A%'
```



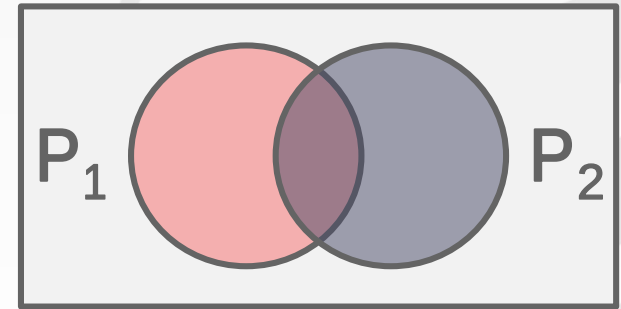
SELECTIONS – COMPLEX PREDICATES

Disjunction:

→ $\text{sel}(P1 \vee P2)$
= $\text{sel}(P1) + \text{sel}(P2) - \text{sel}(P1 \wedge P2)$
= $\text{sel}(P1) + \text{sel}(P2) - \text{sel}(P1) \cdot \text{sel}(P2)$
→ $\text{sel}(\text{age}=2 \text{ OR name LIKE 'A\%'})$

This again assumes that the selectivities are independent.

```
SELECT * FROM people
WHERE age = 2
OR name LIKE 'A%'
```



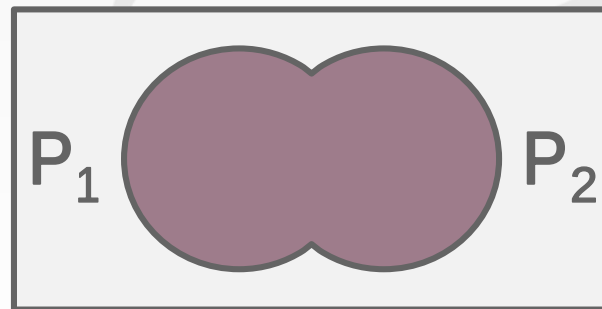
SELECTIONS – COMPLEX PREDICATES

Disjunction:

→ $\text{sel}(P1 \vee P2)$
= $\text{sel}(P1) + \text{sel}(P2) - \text{sel}(P1 \wedge P2)$
= $\text{sel}(P1) + \text{sel}(P2) - \text{sel}(P1) \cdot \text{sel}(P2)$
→ $\text{sel}(\text{age}=2 \text{ OR name LIKE 'A\%'})$

This again assumes that the selectivities are independent.

```
SELECT * FROM people
WHERE age = 2
      OR name LIKE 'A%'
```



SELECTION CARDINALITY

Assumption #1: Uniform Data

→ The distribution of values (except for the heavy hitters) is the same.

Assumption #2: Independent Predicates

→ The predicates on attributes are independent

Assumption #3: Inclusion Principle

→ The domain of join keys overlap such that each key in the inner relation will also exist in the outer table.

CORRELATED ATTRIBUTES

Consider a database of automobiles:

→ # of Makes = 10, # of Models = 100

And the following query:

→ (make="Honda" AND model="Accord")

With the independence and uniformity assumptions, the selectivity is:

→ $1/10 \times 1/100 = 0.001$

But since only Honda makes Accords the real selectivity is $1/100 = 0.01$

COST ESTIMATIONS

Our formulas are nice, but we assume that data values are uniformly distributed.

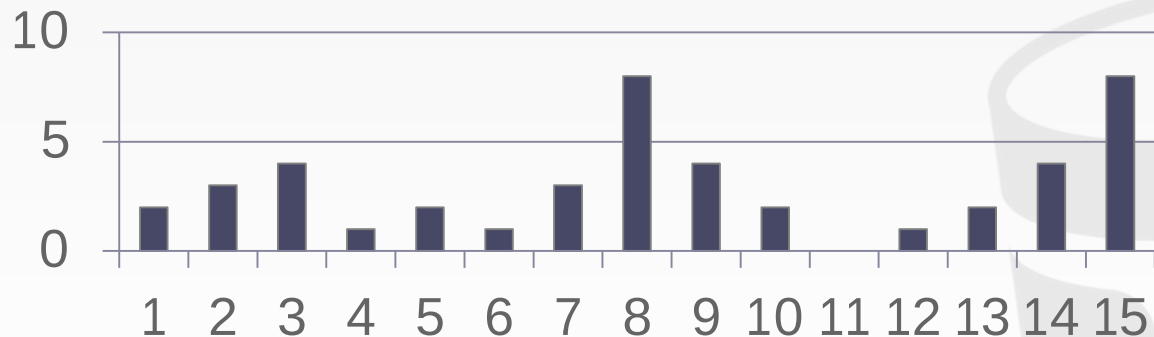
Uniform Approximation



COST ESTIMATIONS

Our formulas are nice, but we assume that data values are uniformly distributed.

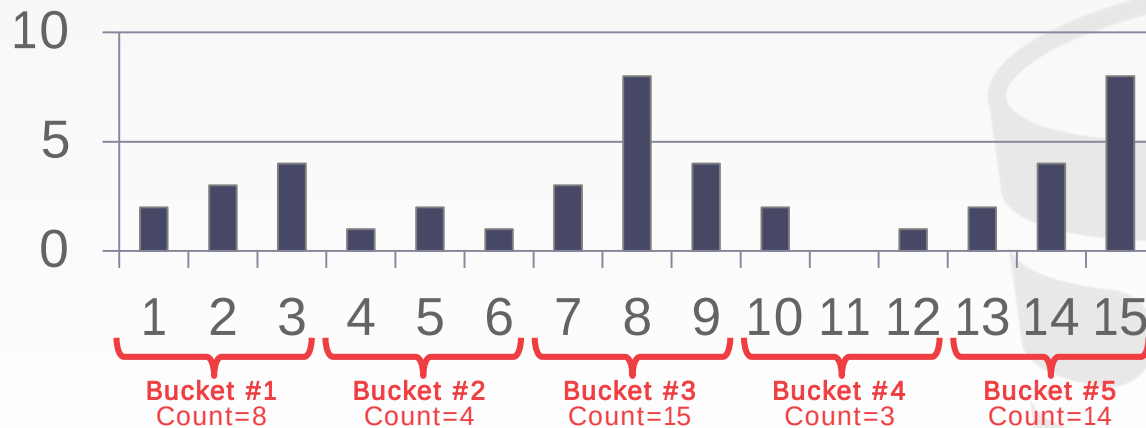
Non-Uniform Approximation



COST ESTIMATIONS

Our formulas are nice, but we assume that data values are uniformly distributed.

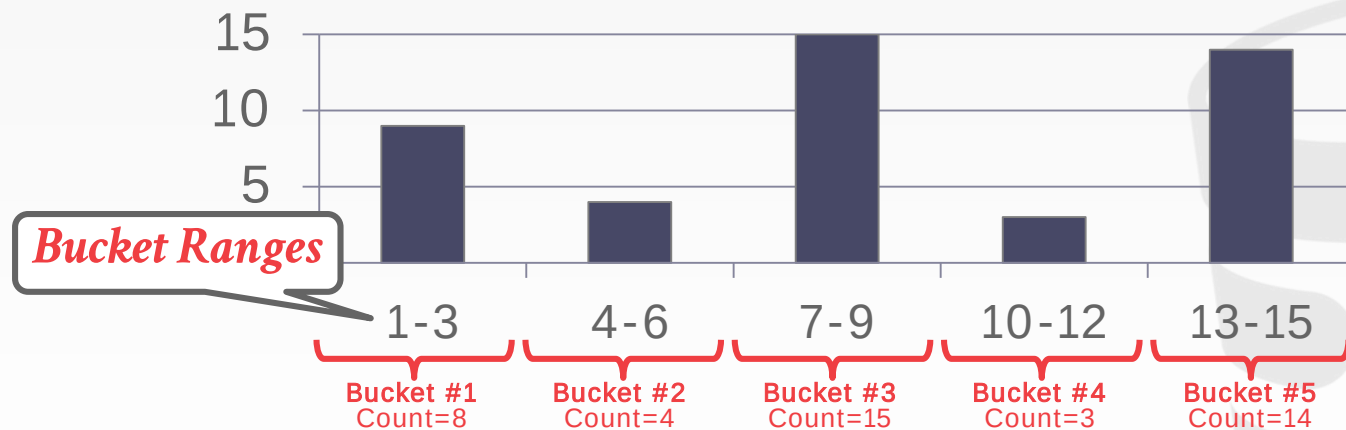
Non-Uniform Approximation



COST ESTIMATIONS

Our formulas are nice, but we assume that data values are uniformly distributed.

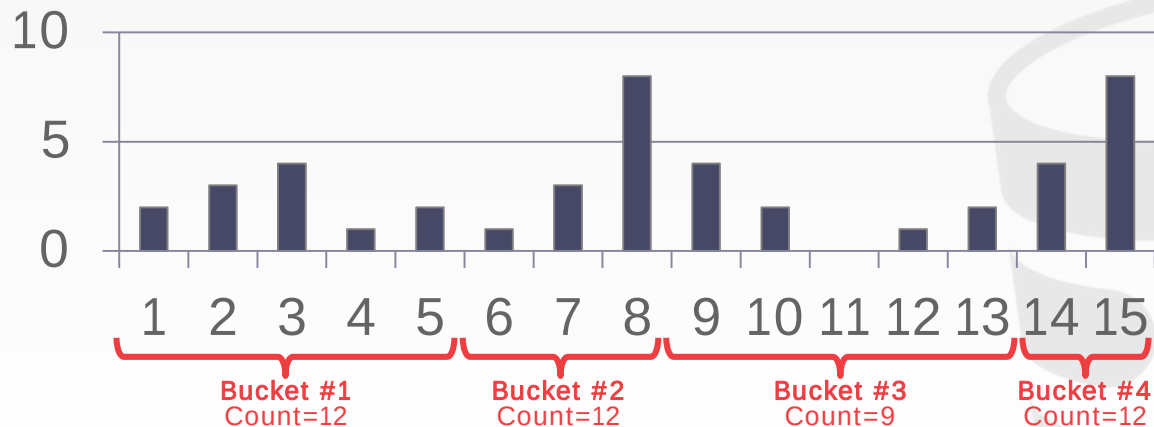
Non-Uniform Approximation



HISTOGRAMS WITH QUANTILES

Vary the width of buckets so that the total number of occurrences for each bucket is roughly the same.

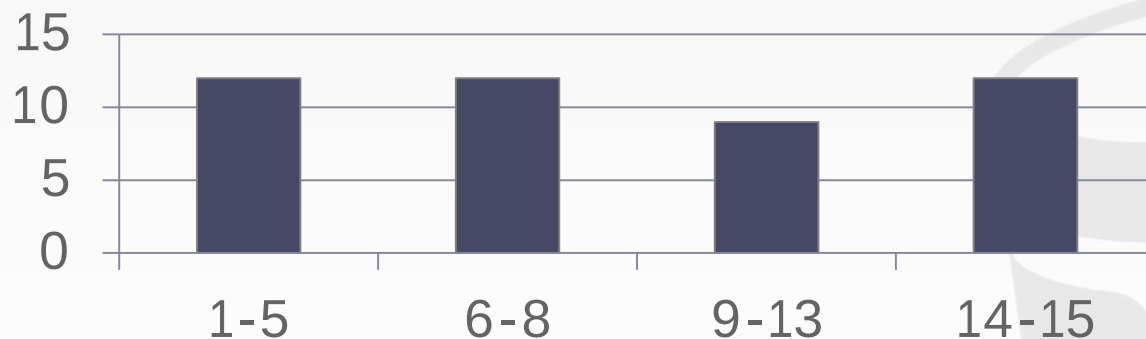
Histogram (Quantiles)



HISTOGRAMS WITH QUANTILES

Vary the width of buckets so that the total number of occurrences for each bucket is roughly the same.

Histogram (Quantiles)



SAMPLING

Modern DBMSs also collect samples from tables to estimate selectivities.

Update samples when the underlying tables changes significantly.

```
SELECT AVG(age)
FROM people
WHERE age > 50
```

id	name	age	status
1001	Obama	58	Rested
1002	Kanye	41	Weird
1003	Tupac	25	Dead
1004	Bieber	25	Crunk
1005	Andy	38	Lit

⋮

1 billion tuples

SAMPLING

Modern DBMSs also collect samples from tables to estimate selectivities.

Update samples when the underlying tables changes significantly.

Table Sample

1001	Obama	58	Rested
1003	Tupac	25	Dead
1005	Andy	38	Lit

```
SELECT AVG(age)
FROM people
WHERE age > 50
```



id	name	age	status
1001	Obama	58	Rested
1002	Kanye	41	Weird
1003	Tupac	25	Dead
1004	Bieber	25	Crunk
1005	Andy	38	Lit

⋮
1 billion tuples

SAMPLING

Modern DBMSs also collect samples from tables to estimate selectivities.

Update samples when the underlying tables changes significantly.

Table Sample

1001	Obama	58	Rested
1003	Tupac	25	Dead
1005	Andy	38	Lit

$\text{sel}(\text{age} > 50) = 1/3$

```
SELECT AVG(age)
FROM people
WHERE age > 50
```



id	name	age	status
1001	Obama	58	Rested
1002	Kanye	41	Weird
1003	Tupac	25	Dead
1004	Bieber	25	Crunk
1005	Andy	38	Lit

⋮
1 billion tuples

OBSERVATION

Now that we can (roughly) estimate the selectivity of predicates, what can we actually do with them?



QUERY OPTIMIZATION

After performing rule-based rewriting, the DBMS will enumerate different plans for the query and estimate their costs.

- Single relation.
- Multiple relations.
- Nested sub-queries.

It chooses the best plan it has seen for the query after exhausting all plans or some timeout.

SINGLE-RELATION QUERY PLANNING

Pick the best access method.

- Sequential Scan
- Binary Search (clustered indexes)
- Index Scan

Predicate evaluation ordering.

Simple heuristics are often good enough for this.

OLTP queries are especially easy...

OLTP QUERY PLANNING

Query planning for OLTP queries is easy because they are **sargable** (**Search **Argument **Able****).**

- It is usually just picking the best index.
- Joins are almost always on foreign key relationships with a small cardinality.
- Can be implemented with simple heuristics.

```
CREATE TABLE people (  
  id INT PRIMARY KEY,  
  val INT NOT NULL,  
  :  
);
```

```
SELECT * FROM people  
WHERE id = 123;
```



MULTI-RELATION QUERY PLANNING

As number of joins increases, number of alternative plans grows rapidly

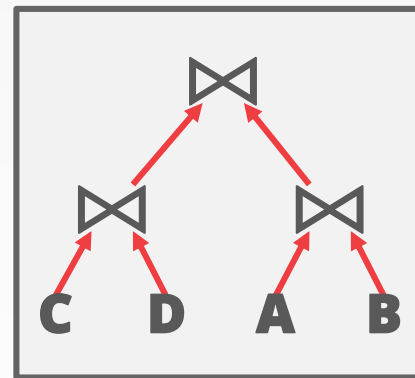
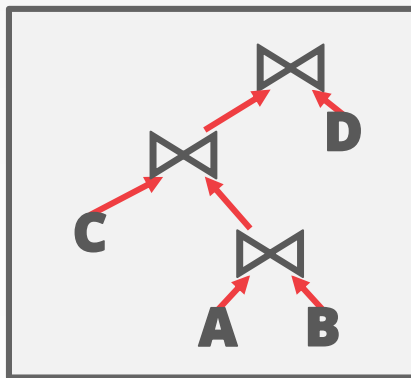
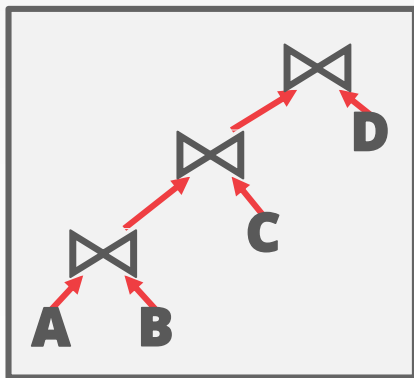
→ We need to restrict search space.

Fundamental decision in **System R**: only left-deep join trees are considered.

→ Modern DBMSs do not always make this assumption anymore.

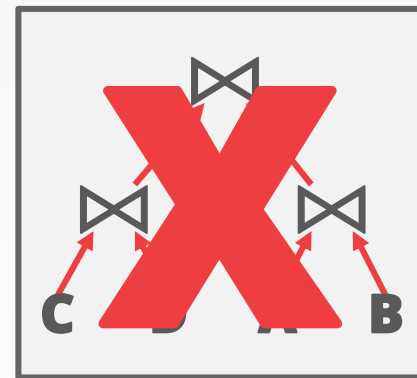
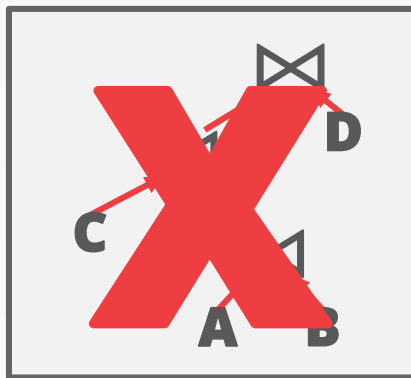
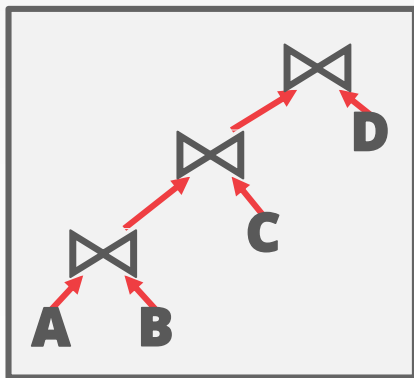
MULTI-RELATION QUERY PLANNING

Fundamental decision in **System R**: Only consider left-deep join trees.



MULTI-RELATION QUERY PLANNING

Fundamental decision in **System R**: Only consider left-deep join trees.



MULTI-RELATION QUERY PLANNING

Fundamental decision in **System R**: Only consider left-deep join trees.

Allows for fully pipelined plans where intermediate results are not written to temp files.
→ Not all left-deep trees are fully pipelined.

MULTI-RELATION QUERY PLANNING

Enumerate the orderings

→ Example: Left-deep tree #1, Left-deep tree #2...

Enumerate the plans for each operator

→ Example: Hash, Sort-Merge, Nested Loop...

Enumerate the access paths for each table

→ Example: Index #1, Index #2, Seq Scan...

Use **dynamic programming** to reduce the number of cost estimations.

DYNAMIC PROGRAMMING

Hash Join

R.a=S.a

Cost:
300

SortMerge Join

R.a=S.a

Cost:
400

SortMerge Join

T.b=S.b

Cost:
280

Hash Join

T.b=S.b

Cost:
200

R ⋈ S
T

R
S
T

T ⋈ S
R
⋮

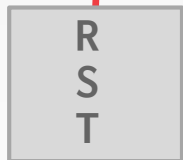
```
SELECT * FROM R, S, T
WHERE R.a = S.a
AND S.b = T.b
```

R ⋈ S ⋈ T

DYNAMIC PROGRAMMING

Hash Join

R.a=S.a Cost: 300



Hash Join

T.b=S.b Cost: 200



```
SELECT * FROM R, S, T
WHERE R.a = S.a
AND S.b = T.b
```



DYNAMIC PROGRAMMING

Hash Join

R.a=S.a Cost: 300

Hash Join

S.b=T.b Cost: 380

```
SELECT * FROM R, S, T
WHERE R.a = S.a
AND S.b = T.b
```

SortMerge Join

S.b=T.b Cost: 400

SortMerge Join

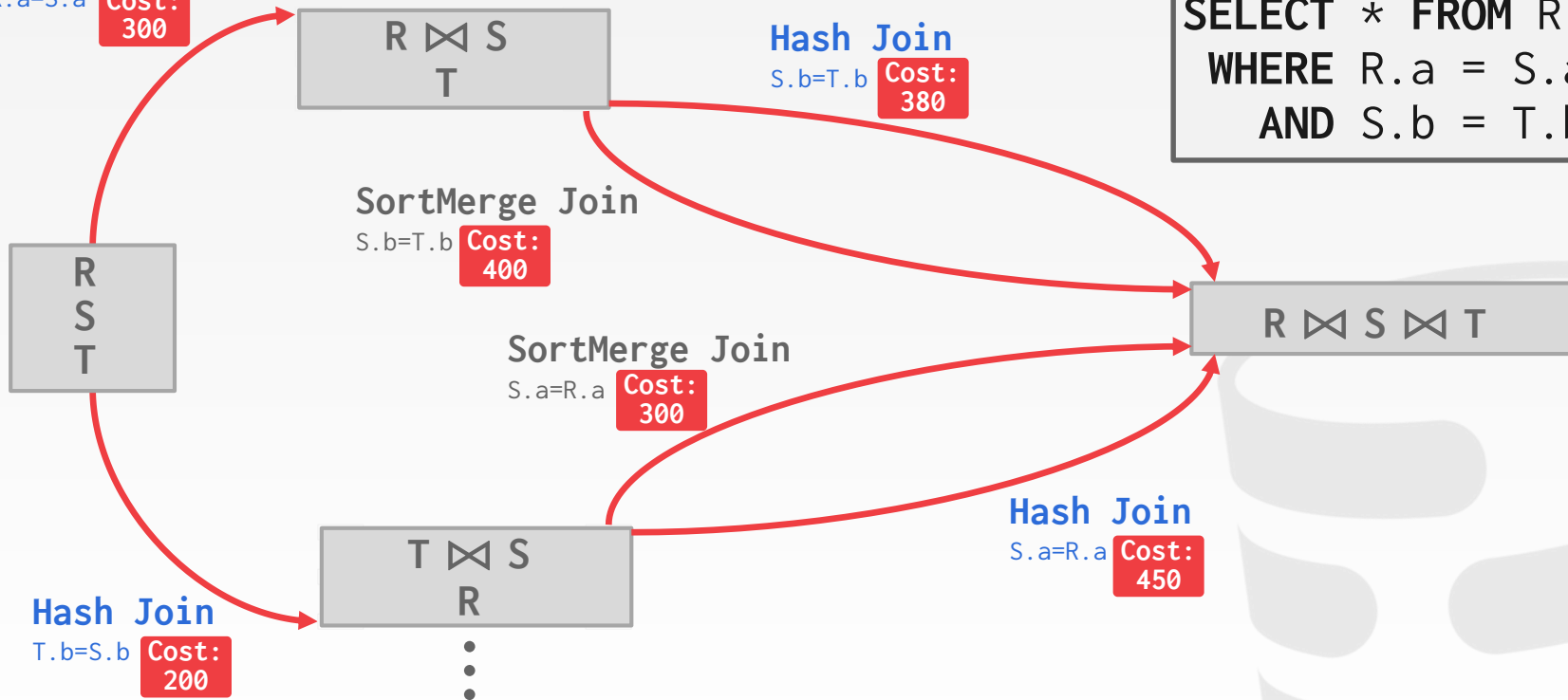
S.a=R.a Cost: 300

Hash Join

S.a=R.a Cost: 450

Hash Join

T.b=S.b Cost: 200



DYNAMIC PROGRAMMING

Hash Join

R.a=S.a **Cost: 300**

Hash Join

S.b=T.b **Cost: 380**

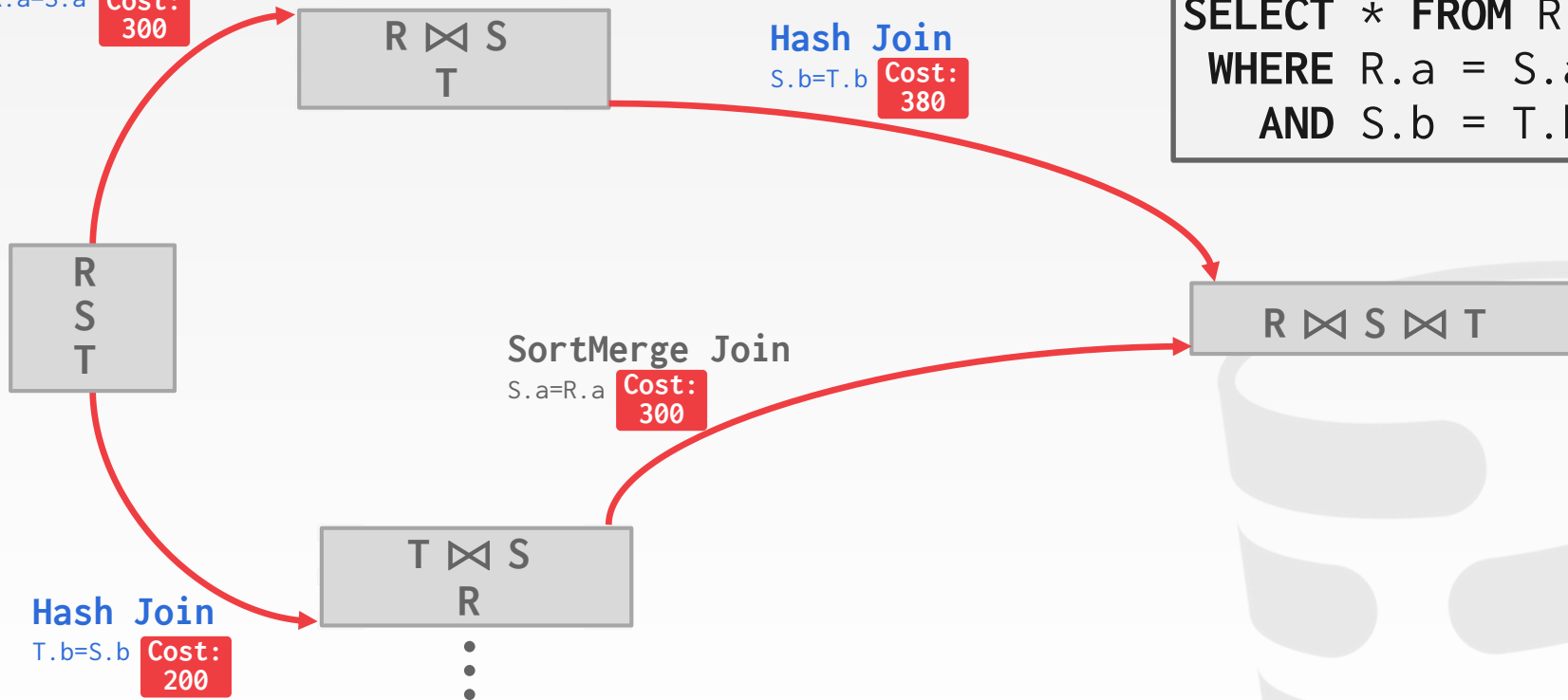
```
SELECT * FROM R, S, T
WHERE R.a = S.a
AND S.b = T.b
```

SortMerge Join

S.a=R.a **Cost: 300**

Hash Join

T.b=S.b **Cost: 200**



DYNAMIC PROGRAMMING

$$\begin{array}{c} R \bowtie S \\ T \end{array}$$

```
SELECT * FROM R, S, T
WHERE R.a = S.a
AND S.b = T.b
```

$$\begin{array}{c} R \\ S \\ T \end{array}$$

SortMerge Join

S.a=R.a **Cost: 300**

Hash Join

T.b=S.b **Cost: 200**

$$\begin{array}{c} T \bowtie S \\ R \\ \vdots \end{array}$$

$$R \bowtie S \bowtie T$$

CANDIDATE PLAN EXAMPLE

How to generate plans for search algorithm:

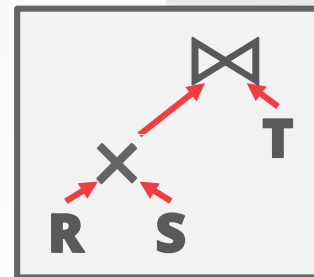
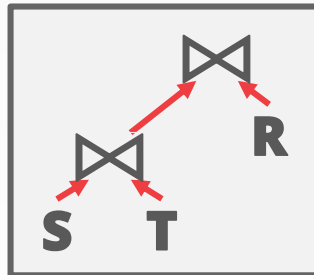
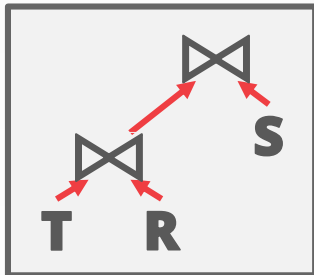
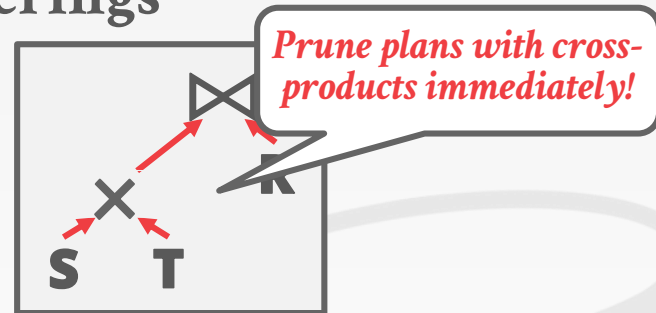
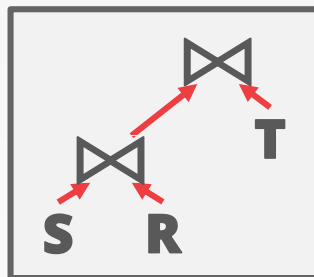
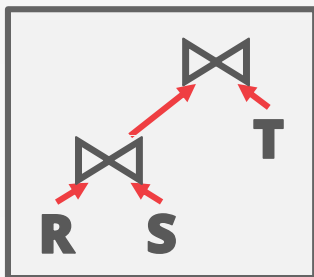
- Enumerate relation orderings
- Enumerate join algorithm choices
- Enumerate access method choices

No real DBMSs does it this way.
It's actually more messy...

```
SELECT * FROM R, S, T  
WHERE R.a = S.a  
AND S.b = T.b
```

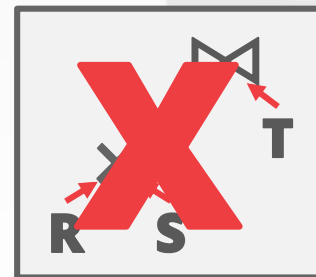
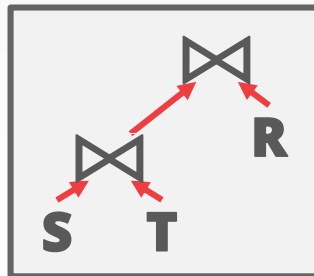
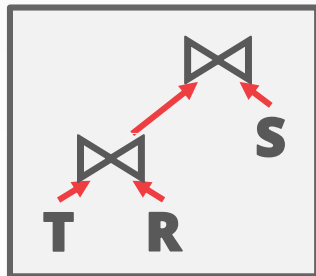
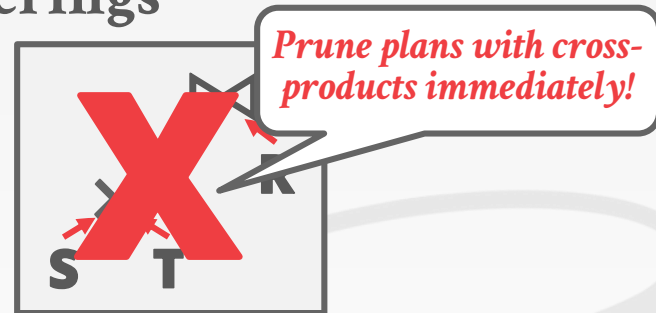
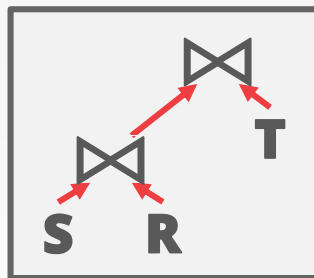
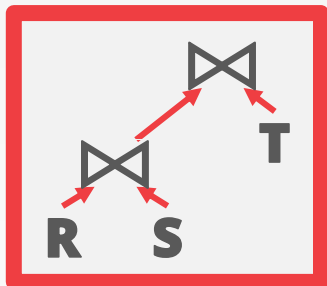
CANDIDATE PLANS

Step #1: Enumerate relation orderings



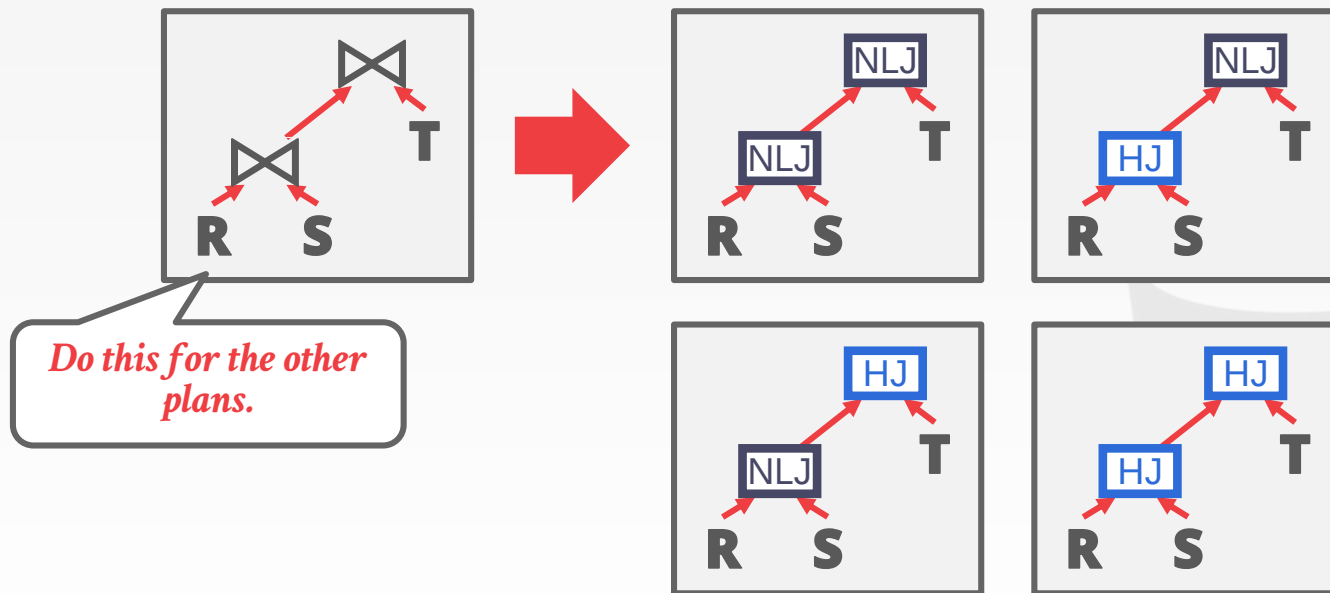
CANDIDATE PLANS

Step #1: Enumerate relation orderings



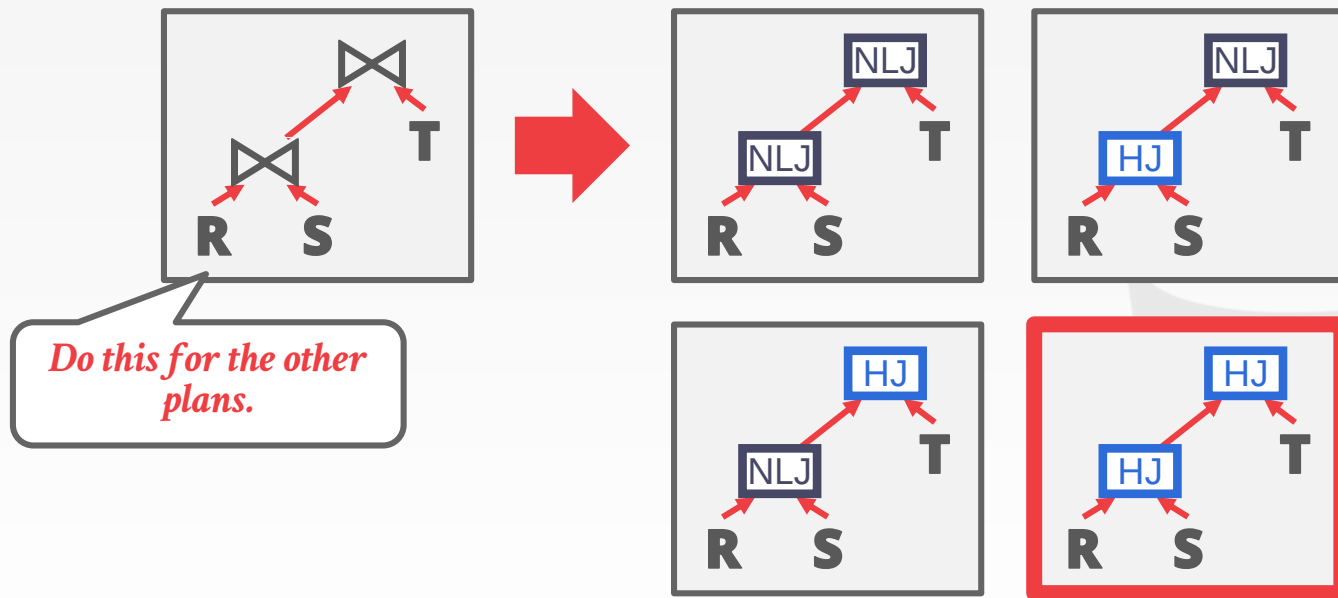
CANDIDATE PLANS

Step #2: Enumerate join algorithm choices



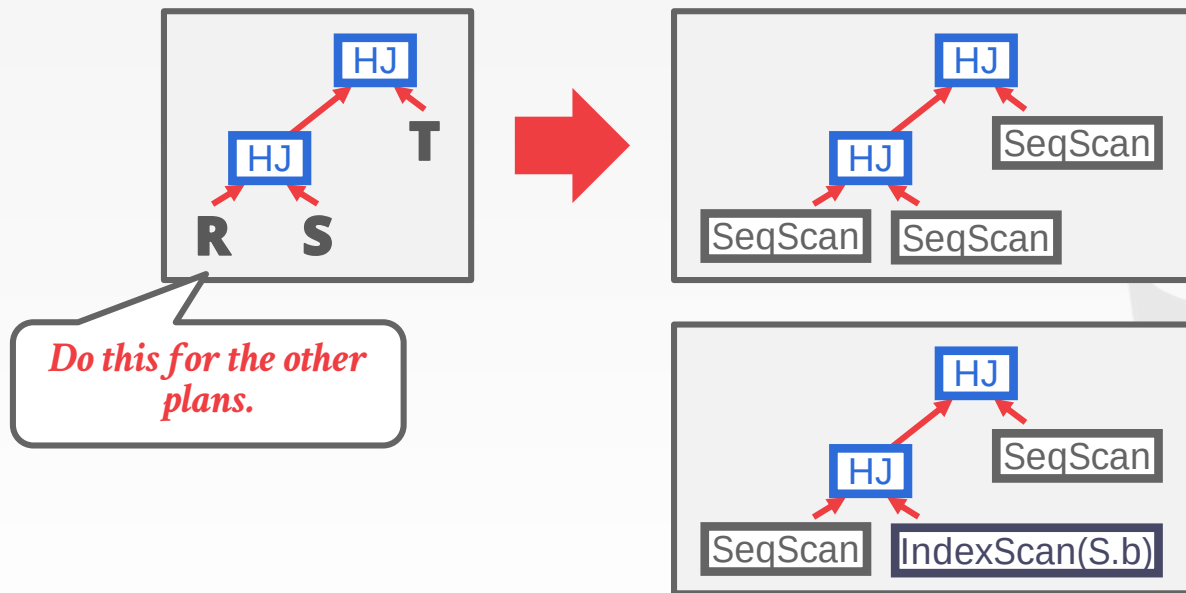
CANDIDATE PLANS

Step #2: Enumerate join algorithm choices



CANDIDATE PLANS

Step #3: Enumerate access method choices



POSTGRES OPTIMIZER

Examines all types of join trees

→ Left-deep, Right-deep, bushy

Two optimizer implementations:

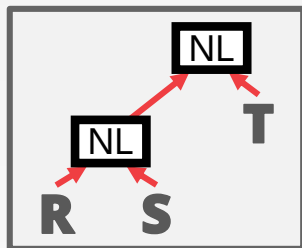
→ Traditional Dynamic Programming Approach

→ Genetic Query Optimizer (GEQO)

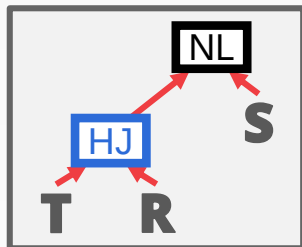
Postgres uses the traditional algorithm when # of tables in query is less than 12 and switches to GEQO when there are 12 or more.

POSTGRES OPTIMIZER

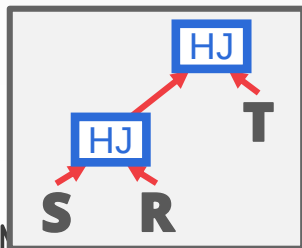
1st Generation



Cost:
300

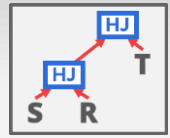


Cost:
200



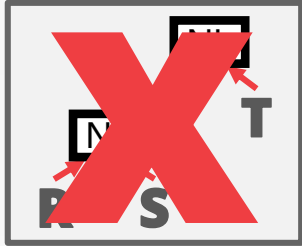
Cost:
100

POSTGRES OPTIMIZER

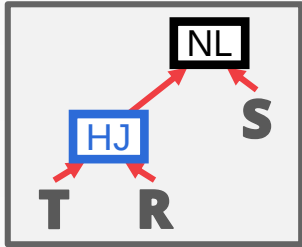


Best: 100

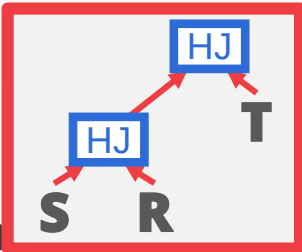
1st Generation



Cost:
300



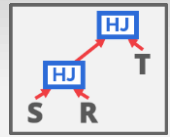
Cost:
200



Cost:
100

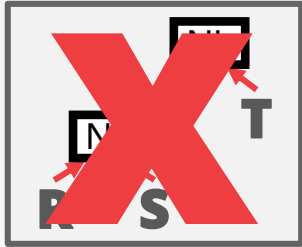


POSTGRES OPTIMIZER

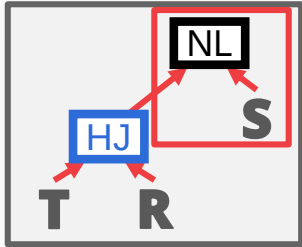


Best: 100

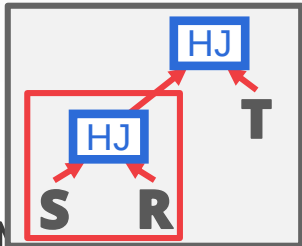
1st Generation



Cost:
300

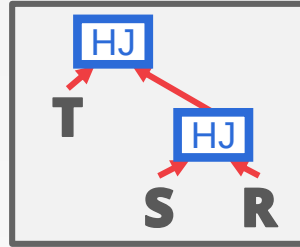


Cost:
200

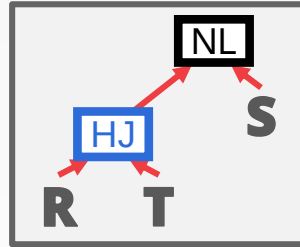


Cost:
100

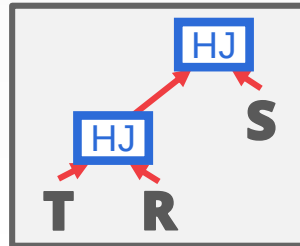
2nd Generation



Cost:
80

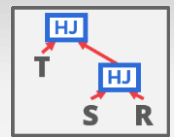


Cost:
200



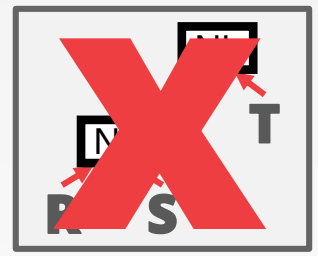
Cost:
110

POSTGRES OPTIMIZER

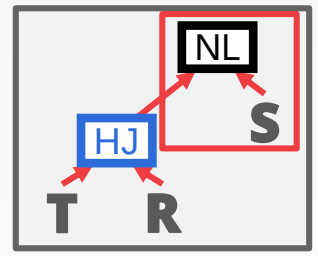


Best: 80

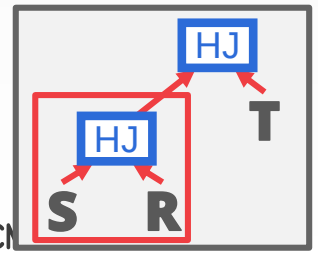
1st Generation



Cost: 300

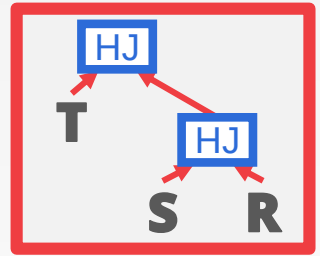


Cost: 200

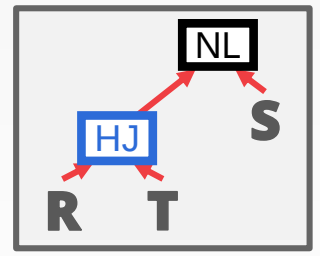


Cost: 100

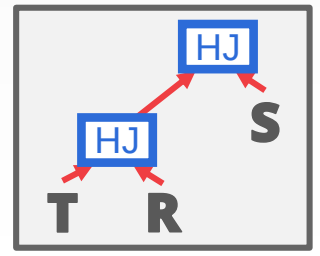
2nd Generation



Cost: 80



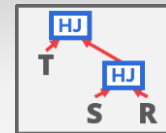
Cost: 200



Cost: 110

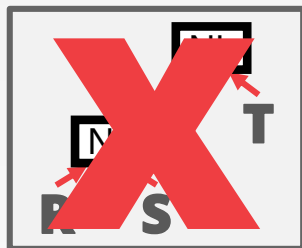


POSTGRES OPTIMIZER

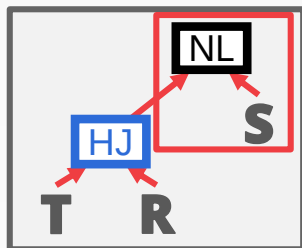


Best: 80

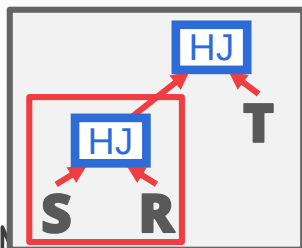
1st Generation



Cost:
300

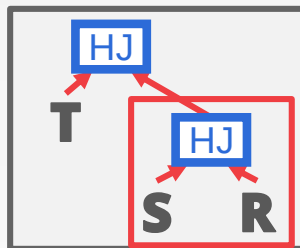


Cost:
200

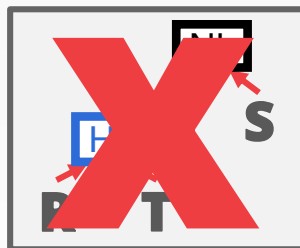


Cost:
100

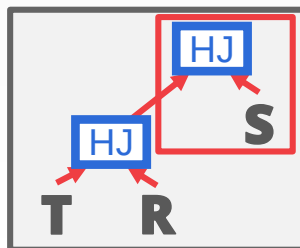
2nd Generation



Cost:
80

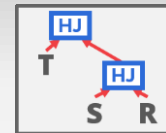


Cost:
200



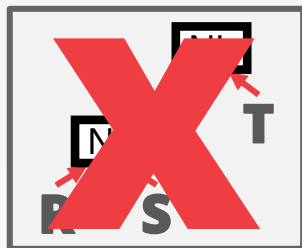
Cost:
110

POSTGRES OPTIMIZER

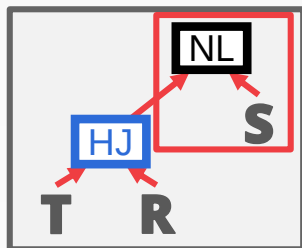


Best: 80

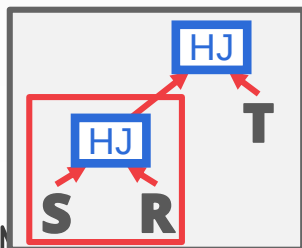
1st Generation



Cost:
300

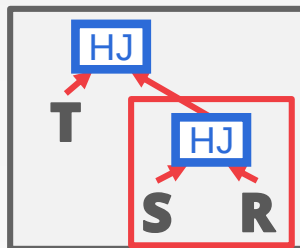


Cost:
200

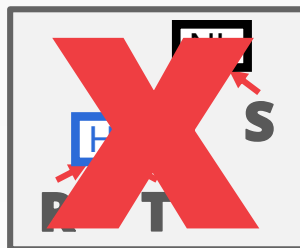


Cost:
100

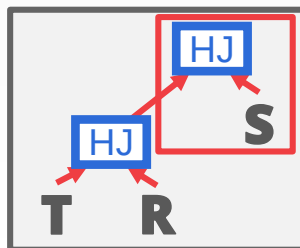
2nd Generation



Cost:
80

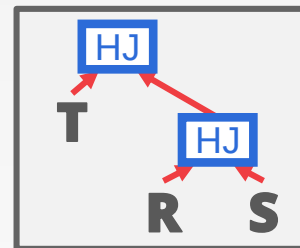


Cost:
200

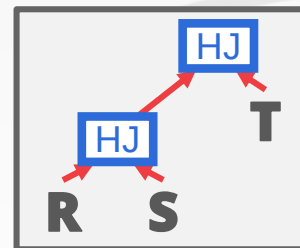


Cost:
110

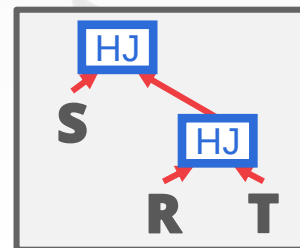
3rd Generation



Cost:
90



Cost:
160



Cost:
120

NESTED SUB-QUERIES

The DBMS treats nested sub-queries in the where clause as functions that take parameters and return a single value or set of values.

Two Approaches:

- Rewrite to de-correlate and/or flatten them
- Decompose nested query and store result to temporary table

NESTED SUB-QUERIES: REWRITE

```
SELECT name FROM sailors AS S
WHERE EXISTS (
  SELECT * FROM reserves AS R
  WHERE S.sid = R.sid
  AND R.day = '2018-10-15'
)
```

NESTED SUB-QUERIES: REWRITE

```
SELECT name FROM sailors AS S
WHERE EXISTS (
  SELECT * FROM reserves AS R
  WHERE S.sid = R.sid
  AND R.day = '2018-10-15'
)
```



```
SELECT name
FROM sailors AS S, reserves AS R
WHERE S.sid = R.sid
AND R.day = '2018-10-15'
```

NESTED SUB-QUERIES: DECOMPOSE

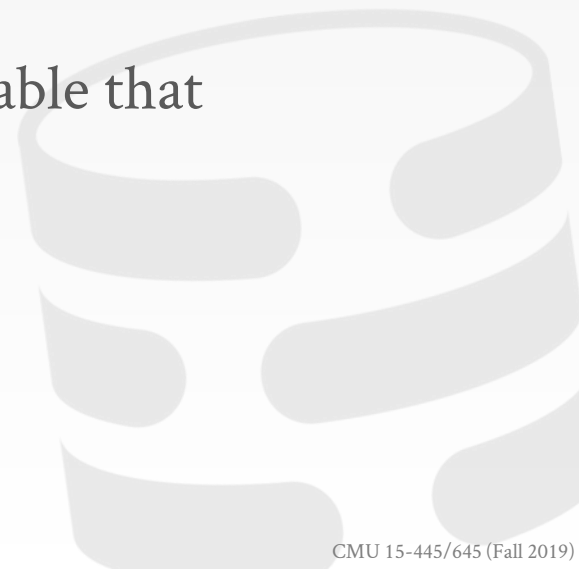
```
SELECT S.sid, MIN(R.day)
  FROM sailors S, reserves R, boats B
 WHERE S.sid = R.sid
   AND R.bid = B.bid
   AND B.color = 'red'
   AND S.rating = (SELECT MAX(S2.rating)
                   FROM sailors S2)
 GROUP BY S.sid
HAVING COUNT(*) > 1
```

For each sailor with the highest rating (over all sailors) and at least two reservations for red boats, find the sailor id and the earliest date on which the sailor has a reservation for a red boat.

DECOMPOSING QUERIES

For harder queries, the optimizer breaks up queries into blocks and then concentrates on one block at a time.

Sub-queries are written to a temporary table that are discarded after the query finishes.



DECOMPOSING QUERIES

```
SELECT S.sid, MIN(R.day)
  FROM sailors S, reserves R, boats B
 WHERE S.sid = R.sid
    AND R.bid = B.bid
    AND B.color = 'red'
    AND S.rating = (SELECT MAX(S2.rating)
                   FROM sailors S2)
 GROUP BY S.sid
HAVING COUNT(*) > 1
```

Nested Block

DECOMPOSING QUERIES

```
SELECT MAX(rating) FROM sailors
```

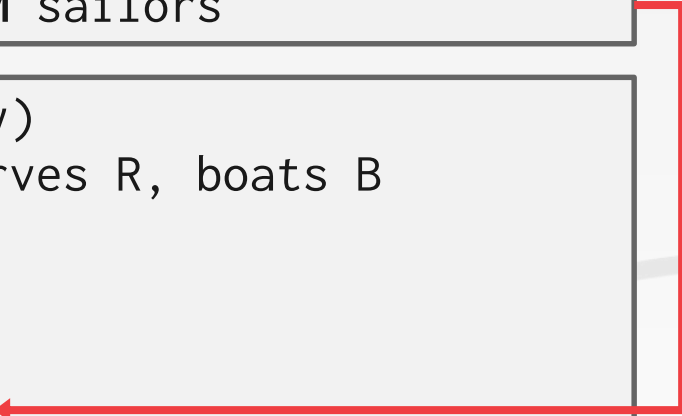
```
SELECT S.sid, MIN(R.day)
  FROM sailors S, reserves R, boats B
 WHERE S.sid = R.sid
   AND R.bid = B.bid
   AND B.color = 'red'
   AND S.rating = (SELECT MAX(S2.rating)
                   FROM sailors S2)
 GROUP BY S.sid
HAVING COUNT(*) > 1
```

Nested Block

DECOMPOSING QUERIES

```
SELECT MAX(rating) FROM sailors
```

```
SELECT S.sid, MIN(R.day)
FROM sailors S, reserves R, boats B
WHERE S.sid = R.sid
      AND R.bid = B.bid
      AND B.color = 'red'
      AND S.rating = ###
GROUP BY S.sid
HAVING COUNT(*) > 1
```



DECOMPOSING QUERIES

```
SELECT MAX(rating) FROM sailors
```

```
SELECT S.sid, MIN(R.day)
  FROM sailors S, reserves R, boats B
 WHERE S.sid = R.sid
   AND R.bid = B.bid
   AND B.color = 'red'
   AND S.rating = ### ←
 GROUP BY S.sid
HAVING COUNT(*) > 1
```

Outer Block

CONCLUSION

Filter early as possible.

Selectivity estimations

- Uniformity
- Independence
- Histograms
- Join selectivity

Dynamic programming for join orderings

Rewrite nested queries

Again, query optimization is hard...



EXTRA CREDIT

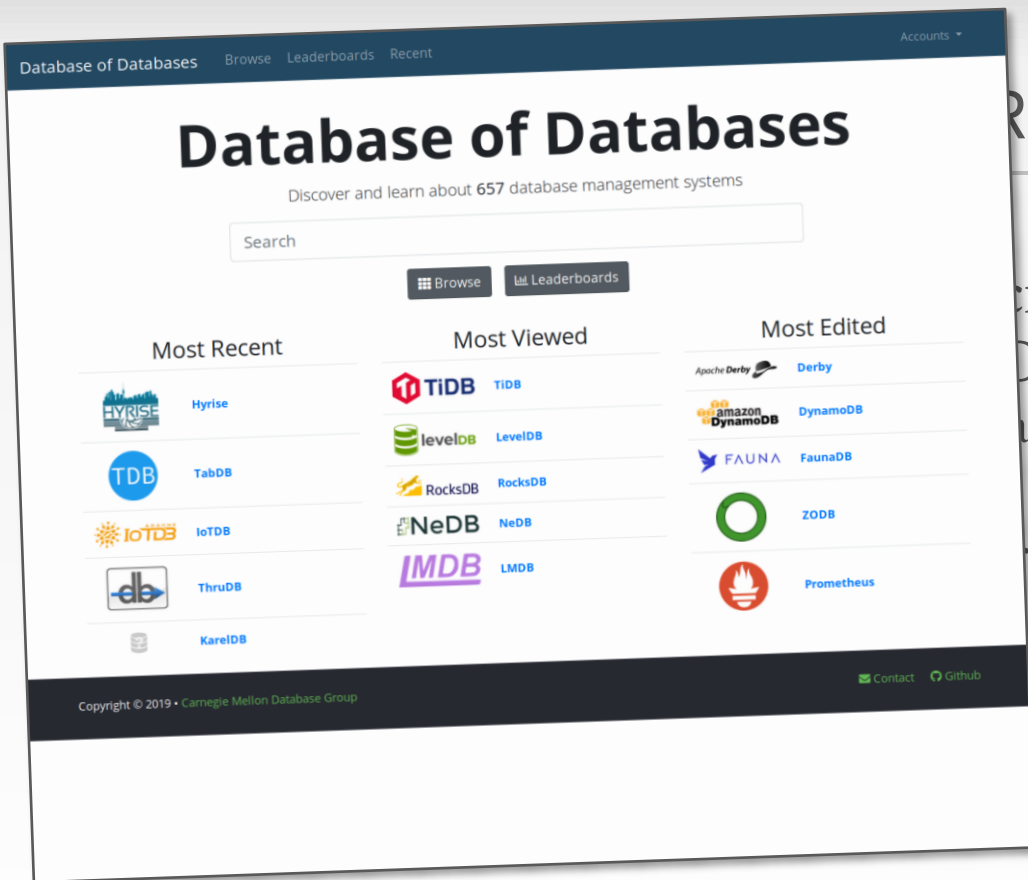
Each student can earn extra credit if they write a encyclopedia article about a DBMS.

→ Can be academic/commercial, active/historical.

Each article will use a standard taxonomy.

→ For each feature category, you select pre-defined options for your DBMS.

→ You will then need to provide a summary paragraph with citations for that category.



CREDIT

credit if they write a DBMS.

active/historical.

and taxonomy.

select pre-defined options

summary paragraph with

Database of Databases Browse Leaderboards Recent Accounts

Database of Databases

Discover a database

Search

Most Recent

-  Hyrise
-  TDB
-  IoTDB
-  ThruDB
-  KarelDB

Copyright © 2019 • Carnegie Mellon Database Group

Database of Databases Browse Leaderboards Recent Accounts

Refine by

Start Year Enable

End Year Enable

Country

- ☐ Australia
- ☐ Austria
- ☐ Bangladesh

[Show more](#)

Compatible With

- ☐ Access
- ☐ Caché
- ☐ Cassandra

[Show more](#)

Embeds / Uses

- ☐ Berkeley DB
- ☐ BoltDB
- ☐ Cassandra

[Show more](#)

Derived From

- ☐ Accumulo
- ☐ Adaptive Server Enterprise
- ☐ Btrieve

[Show more](#)

Inspired By

- ☐ BigQuery
- ☐ C-Store
- ☐ Calvin

[Show more](#)

Operating System

- ☐ AIX
- ☐ All OS with Java VM
- ☐ Android

[Show more](#)

Programming Languages

- ☐ ActionScript
- ☐ Assembly
- ☐ Bash
- ☒ Rust

[Show more](#)

Project Types

Search

Begin searching!

Search

Found 8 databases

-  **IndraDB**
Last updated July 18, 2019, 6:34 p.m.
-  **Mentat**
Last updated July 19, 2019, 12:11 a.m.
-  **Sled**
Last updated May 14, 2019, 10:43 p.m.
-  **LlamaDB**
Last updated May 16, 2018, 7:20 p.m.
-  **PumpkinDB**
Last updated Dec. 30, 2018, 10:04 a.m.
-  **TerminusDB**
Last updated Sept. 24, 2019, noon
-  **TiKV**
Last updated July 18, 2019, 5:59 p.m.
-  **LocustDB**
Last updated July 18, 2019, 6:42 p.m.

write a

ed options

graph with

Database of Databases

Discover a

Search

Most Recent



Hyrise



TabDB



IoTDB



ThruDB



KarelDB

Copyright © 2019 Carnegie Mellon Database Group

Search

Refine by

Start Year

Enable

End Year

Enable

Country

- ☐ Australia
- ☐ Austria
- ☐ Bangladesh

[Show more](#)

Compatible With

- ☐ Access
- ☐ Caché
- ☐ Cassandra

[Show more](#)

Embeds / Uses

- ☐ Berkeley DB
- ☐ BoltDB
- ☐ Cassandra

[Show more](#)

Derived From

- ☐ Accumulo
- ☐ Adaptive Server Enterprise
- ☐ Btrieve

[Show more](#)

Inspired By

- ☐ BigQuery
- ☐ C-Store
- ☐ Calvin

[Show more](#)

Operating System

- ☐ AIX
- ☐ All OS with Java VM
- ☐ Android

[Show more](#)

Programming Languages

- ☐ ActionScript
- ☐ Assembly
- ☐ Bash
- ☒ Rust

[Show more](#)

Project Types

Begin searching!



IndraDB

Last updated July 18, 2019, 6:34 p.m.



LlamaDB

Last updated May 16, 2018, 7:20 p.m.



LocustDB

Last updated July 18, 2019, 6:42 p.m.

PumpkinDB

PumpkinDB is a low-level event sourcing database engine that is ACID-compliant. It is a database engine that could be used to build different types of event sourcing systems such as embedded and client-server ones.

PumpkinDB is designed to be immutable, the reason behind this is that overwriting data could be unsafe, valuable history of data will be erased. As the cost of storage dropping, a more effective way of managing data is to enforcing immutability of key's value. And writing to the database is close to functional, such that writing side doesn't have to wait for shared resources, the read side will figure out the correct result.

In order to have control over querying costs, it provides an embedded executable imperative language, PumpkinScript, which is a low-level untyped language inspired by MUMPS.

PumpkinDB does not have custom protocols for communication, instead, it has a pipeline to a script executor. When the applications need to communicate with PumpkinDB, small PumpkinScript programs are sent through a network interface in order to do that.

History

PumpkinDB is a descendant of an event capture and querying framework ES4j. The difference from ES4j is that PumpkinDB has a HLC timestamp, a UUID, complies with the ELF format, and it treats events as binary blobs. It started as a backend for a lazy event sourcing approach on 2017.

Concurrency Control

Not Supported

Data Model

Key/Value

It supports binary keys and values, which enables the use of any encoding such as XML, JSON and Protobuf.



Website

<http://pumpkindb.org/>

Source Code

<https://github.com/PumpkinDB/PumpkinDB>

Tech Docs

<http://pumpkindb.org/doc/>

Developer

Yurii Rashkovskii

Country of Origin

CA

Start Year

2017

Project Type

Open Source

Written in

Rust

Operating Systems

Linux, Windows

Licenses

Mozilla Public License

DBDB.IO

All the articles will be hosted on dbdb.io

→ I will post registration details on Piazza.

I will post a sign-up sheet for you to pick what DBMS you want to write about.

- If you choose a widely known DBMS, then the article will need to be comprehensive.
- If you choose an obscure DBMS, then you will have to do the best you can to find information.

All the articles will be
→ I will post registration

I will post a sign-up s
DBMS you want to v
→ If you choose a widely
need to be comprehen
→ If you choose an obs
the best you can to fi

Database of Databases [Browse](#) [Leaderboards](#) [Recent](#) [Create](#) [Empty Fields](#) [pavlo](#)

Edit Database System

Name

Logo
Currently: [logos/pumpkindb.png](#)
Change: No file selected.

Url
URL of the DBMS company or project

Developer
The original organization that developed the DBMS.

Source Code URL
URL of where to download source code (if available)

Tech docs
URL of the where to find technical documentation about the DBMS

Wikipedia URL
URL of Wikipedia article about this system (if available)

Project Type
Academic
Commercial
Industrial Research
Open Source

Start year
End year

Start year citations
Separate the urls with commas

End year citations
Separate the urls with commas

Former names
Previous names of the system

Acquired by
Name of the company that first acquired the DBMS

Acquired by citations
Separate the urls with commas

Countries of Origin
Country of where the DBMS company or project started

Revision Comment



HOW TO DECIDE

Pick a DBMS based on whatever criteria you want:

- Country of Origin
- Popularity
- Programming Language
- Single-Node vs. Embedded vs. Distributed
- Disk vs. Memory
- Row Store vs. Column Store
- Open-Source vs. Proprietary



☠️ PLAGIARISM WARNING ☠️

This article must be your own writing with your own images. You may **not** copy text/images directly from papers or other sources that you find on the web.

Plagiarism will **not** be tolerated.

See [CMU's Policy on Academic Integrity](#) for additional information.

NEXT CLASS

Transactions!

→ aka the second hardest part about database systems



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]